

```

# BLOB Principle - Caracterisation de la transition de regime
# Script rigoureux ASCII pur pour Colab
# Trois protocoles complementaires en un script :
#   A. Localisation fine de la transition (N entre 25k et 50k)
#   B. Test physique : dilution Rips via scan kappa_0 a N=50000 fixe
#   C. Convergence variable : test thermalisation a N=50000 fixe
#
# Total estime : 2-3 heures sur Colab CPU

import numpy as np
import time
from collections import defaultdict

# =====
# Parametres canoniques
# =====
KAPPA_0_DEFAULT = 8
BETA = 0.30
TAU = 0.50

# =====
# Classe BlobGraph (identique aux campagnes precedentes, ASCII pur)
# =====
class BlobGraph:
    def __init__(self, N, seed, is_null=False, kappa_0=None):
        self.N = N
        self.is_null = is_null
        self.kappa_0 = kappa_0 if kappa_0 is not None else KAPPA_0_DEFAULT
        self.rng = np.random.default_rng(seed)
        self.E_target = int(N * self.kappa_0 / 2)
        self.adj = [set() for _ in range(N)]
        self.edge_list = []
        self._init_graph()
        self.deg = np.array([len(self.adj[i]) for i in range(N)], dtype=float)
        self.n_tri = self._count_triangles()
        self.S_curr = self._compute_S(self.n_tri)

    def _init_graph(self):
        for i in range(self.N):
            u, v = i, (i + 1) % self.N
            if v not in self.adj[u]:
                self.adj[u].add(v)
                self.adj[v].add(u)
                self.edge_list.append((min(u, v), max(u, v)))

```

```

while len(self.edge_list) < self.E_target:
    u = int(self.rng.integers(self.N))
    v = int(self.rng.integers(self.N))
    if u != v and v not in self.adj[u]:
        self.adj[u].add(v)
        self.adj[v].add(u)
        self.edge_list.append((min(u, v), max(u, v)))

def _count_triangles(self):
    tri = sum(len(self.adj[u] & self.adj[v]) for u, v in self.edge_list)
    return tri // 3

def _compute_S(self, n_tri_val):
    mu = self.deg.mean()
    D = self.deg.std() / mu if mu > 0 else 0.0
    return np.log(1 + n_tri_val) - BETA * D

def step_blob(self):
    idx = self.rng.integers(len(self.edge_list))
    u_rm, v_rm = self.edge_list[idx]
    u_add = int(self.rng.integers(self.N))
    v_add = int(self.rng.integers(self.N))
    if u_add == v_add or v_add in self.adj[u_add]:
        return False
    u_add, v_add = min(u_add, v_add), max(u_add, v_add)
    if (u_add, v_add) == (u_rm, v_rm):
        return False
    dt_rm = len(self.adj[u_rm] & self.adj[v_rm])
    self.adj[u_rm].discard(v_rm)
    self.adj[v_rm].discard(u_rm)
    self.deg[u_rm] -= 1
    self.deg[v_rm] -= 1
    dt_add = len(self.adj[u_add] & self.adj[v_add])
    self.adj[u_add].add(v_add)
    self.adj[v_add].add(u_add)
    self.deg[u_add] += 1
    self.deg[v_add] += 1
    new_n_tri = self.n_tri - dt_rm + dt_add
    if new_n_tri < 0:
        self.adj[u_add].discard(v_add)
        self.adj[v_add].discard(u_add)
        self.deg[u_add] -= 1
        self.deg[v_add] -= 1
        self.adj[u_rm].add(v_rm)
        self.adj[v_rm].add(u_rm)
        self.deg[u_rm] += 1
        self.deg[v_rm] += 1

```

```

        return False
new_S = self._compute_S(new_n_tri)
log_alpha = self.N * (new_S - self.S_curr) / TAU
if log_alpha >= 0:
    accepted = True
else:
    accepted = self.rng.random() < np.exp(max(log_alpha, -500))
if accepted:
    self.edge_list[idx] = (u_add, v_add)
    self.n_tri = new_n_tri
    self.S_curr = new_S
    return True
else:
    self.adj[u_add].discard(v_add)
    self.adj[v_add].discard(u_add)
    self.deg[u_add] -= 1
    self.deg[v_add] -= 1
    self.adj[u_rm].add(v_rm)
    self.adj[v_rm].add(u_rm)
    self.deg[u_rm] += 1
    self.deg[v_rm] += 1
    return False

def step_null(self):
    idx = self.rng.integers(len(self.edge_list))
    u_rm, v_rm = self.edge_list[idx]
    u_add = int(self.rng.integers(self.N))
    v_add = int(self.rng.integers(self.N))
    if u_add == v_add or v_add in self.adj[u_add]:
        return False
    u_add, v_add = min(u_add, v_add), max(u_add, v_add)
    if (u_add, v_add) == (u_rm, v_rm):
        return False
    dt_rm = len(self.adj[u_rm] & self.adj[v_rm])
    self.adj[u_rm].discard(v_rm)
    self.adj[v_rm].discard(u_rm)
    self.deg[u_rm] -= 1
    self.deg[v_rm] -= 1
    dt_add = len(self.adj[u_add] & self.adj[v_add])
    self.adj[u_add].add(v_add)
    self.adj[v_add].add(u_add)
    self.deg[u_add] += 1
    self.deg[v_add] += 1
    new_n_tri = self.n_tri - dt_rm + dt_add
    if new_n_tri < 0:
        self.adj[u_add].discard(v_add)
        self.adj[v_add].discard(u_add)

```

```

        self.deg[u_add] -= 1
        self.deg[v_add] -= 1
        self.adj[u_rm].add(v_rm)
        self.adj[v_rm].add(u_rm)
        self.deg[u_rm] += 1
        self.deg[v_rm] += 1
        return False

    self.edge_list[idx] = (u_add, v_add)
    self.n_tri = new_n_tri
    self.S_curr = self._compute_S(new_n_tri)
    return True

# =====
# Fonction de simulation generique
# =====
def run_one(N, seed, burn_in, production, is_null, kappa_0=None, label_prefix='')
    t_start = time.time()
    type_label = 'NULL' if is_null else 'BLOB'
    k_label = '' if kappa_0 is None else ' k0=' + str(kappa_0)
    log_id = ' [' + label_prefix + type_label + ' N=' + str(N) + ' s=' + str(seed)
    print(log_id + ' Start...', flush=True)

    sim = BlobGraph(N=N, seed=seed, is_null=is_null, kappa_0=kappa_0)
    ntri_init = sim.n_tri
    step_func = sim.step_null if is_null else sim.step_blob

    t_burn = time.time()
    acc_burn = 0
    cks = [burn_in // 4, burn_in // 2, 3 * burn_in // 4]
    for i in range(burn_in):
        if step_func():
            acc_burn += 1
        if i + 1 in cks:
            ac = acc_burn / (i + 1)
            msg = ' burn ' + str(i+1) + '/' + str(burn_in) + ' acc=' + '{:.3f}'
            msg += ' n_tri=' + str(sim.n_tri) + ' t=' + '{:.0f}'.format(time.time)
            print(msg, flush=True)

    acc_prod = 0
    ntri_samples = []
    S_samples = []
    D_samples = []
    stride = max(1, production // 200)
    for i in range(production):
        if step_func():
            acc_prod += 1

```

```

        if i % stride == 0:
            ntri_samples.append(sim.n_tri)
            S_samples.append(sim.S_curr)
            mu = sim.deg.mean()
            D_samples.append(sim.deg.std() / mu if mu > 0 else 0.0)

acc_rate = acc_prod / production
ntri_arr = np.array(ntri_samples)
S_arr = np.array(S_samples)
D_arr = np.array(D_samples)
mid = len(S_arr) // 2
drift = abs(S_arr[mid:].mean() - S_arr[:mid].mean()) / abs(S_arr[:mid].mean())
total = time.time() - t_start

msg = log_id + ' Done. acc=' + '{:.3f}'.format(acc_rate)
msg += ' n_tri_final=' + str(sim.n_tri)
msg += ' n_tri_mean=' + '{:.1f}'.format(ntri_arr.mean())
msg += ' drift=' + '{:.3%}'.format(drift)
msg += ' t=' + '{:.0f}'.format(total) + 's'
print(msg, flush=True)

return {
    'type': type_label,
    'N': N,
    'seed': seed,
    'kappa_0': sim.kappa_0,
    'burn_in': burn_in,
    'production': production,
    'ntri_init': int(ntri_init),
    'ntri_final': int(sim.n_tri),
    'ntri_mean': float(ntri_arr.mean()),
    'ntri_std': float(ntri_arr.std()),
    'acc_prod': float(acc_rate),
    'S_per_N_mean': float(S_arr.mean() / N),
    'D_mean': float(D_arr.mean()),
    'D_std': float(D_arr.std()),
    'drift': float(drift),
    'time_sec': float(total),
}

# =====
# Synthèse statistique
# =====
def aggregate(results, group_key='N'):
    groups = defaultdict(list)
    for r in results:

```

```

        groups[r[group_key]].append(r)
out = {}
for k in sorted(groups.keys()):
    rs = groups[k]
    out[k] = {
        'n_seeds': len(rs),
        'ntri_mean': float(np.mean([r['ntri_mean'] for r in rs])),
        'ntri_std': float(np.std([r['ntri_mean'] for r in rs])),
        'acc_mean': float(np.mean([r['acc_prod'] for r in rs])),
        'D_mean': float(np.mean([r['D_mean'] for r in rs])),
        'drift_mean': float(np.mean([r['drift'] for r in rs])),
    }
return out

def compute_ratio_snr(blob_agg, null_agg):
    out = {}
    for N in blob_agg:
        if N not in null_agg:
            continue
        b = blob_agg[N]
        nu = null_agg[N]
        ratio = b['ntri_mean'] / nu['ntri_mean'] if nu['ntri_mean'] > 0 else 0
        # SNR : ecart en sigma null
        snr = abs(b['ntri_mean'] - nu['ntri_mean']) / nu['ntri_std'] if nu['ntri_std'] > 0 else 0
        out[N] = {'ratio': ratio, 'snr': snr,
                  'blob_ntri': b['ntri_mean'], 'null_ntri': nu['ntri_mean'],
                  'blob_std': b['ntri_std'], 'null_std': nu['ntri_std']}
    return out

# =====
# PROTOCOLE A : Localisation fine de la transition
# =====
def protocol_A():
    print('')
    print('=' * 75)
    print('  PROTOCOLE A : Localisation fine de la transition de regime')
    print('  N entre 25k et 50k, 3 seeds par N')
    print('  But : determiner si transition nette (phenomene critique)')
    print('          ou crossover lisse')
    print('=' * 75)

    N_LIST_A = [27500, 30000, 32500, 35000, 37500, 40000, 45000]
    N_SEEDS_A = 3
    BURN_FACTOR_A = 5
    PROD_FACTOR_A = 5

```

```

print('  Tailles : ' + str(N_LIST_A))
print('  Seeds par N et par type : ' + str(N_SEEDS_A))
print('  Burn = ' + str(BURN_FACTOR_A) + ' x N, Prod = ' + str(PROD_FACTOR_A))
print('  Total : ' + str(len(N_LIST_A) * N_SEEDS_A * 2) + ' simulations')
print('  Duree estimee : 30-50 minutes')
print('=' * 75)

results_A = []
for N in N_LIST_A:
    burn = BURN_FACTOR_A * N
    prod = PROD_FACTOR_A * N
    print('')
    print('--- N = ' + str(N) + ' ---')
    for seed in range(N_SEEDS_A):
        r = run_one(N, seed, burn, prod, is_null=False, label_prefix='A.')
        results_A.append(r)
    for seed in range(N_SEEDS_A):
        r = run_one(N, seed, burn, prod, is_null=True, label_prefix='A.')
        results_A.append(r)

blob_A = aggregate([r for r in results_A if r['type'] == 'BLOB'], 'N')
null_A = aggregate([r for r in results_A if r['type'] == 'NULL'], 'N')
ratios_A = compute_ratio_snr(blob_A, null_A)

print('')
print('=' * 75)
print('  RESULTATS PROTOCOLE A')
print('=' * 75)
print('  ' + 'N'.ljust(10) + 'n_tri_blob'.ljust(14) + 'n_tri_null'.ljust(14)
      + 'ratio'.ljust(10) + 'SNR'.ljust(12) + 'D_blob')
print('  ' + '-' * 70)
for N in sorted(ratios_A.keys()):
    r = ratios_A[N]
    line = '  ' + str(N).ljust(10)
    line += '{:.1f}'.format(r['blob_ntri']).ljust(14)
    line += '{:.1f}'.format(r['null_ntri']).ljust(14)
    line += '{:.2f}'.format(r['ratio']).ljust(10)
    line += '{:.1f}sigma'.format(r['snr']).ljust(12)
    line += '{:.4f}'.format(blob_A[N]['D_mean'])
    print(line)

return results_A, blob_A, null_A, ratios_A

# =====
# PROTOCOLE B : Test physique - dilution Rips via scan kappa_0

```

```

# =====
def protocol_B():
    print('')
    print('=' * 75)
    print(' PROTOCOLE B : Test physique - dilution Rips')
    print(' N=50000 fixe, scan kappa_0 in {6, 8, 12, 16, 24}')
    print(' But : verifier hypothese dilution Rips')
    print('          si n_tri_blob remonte avec kappa_0 -> hypothese confirmee')
    print('=' * 75)

    N_FIXED = 50000
    KAPPA_LIST = [6, 8, 12, 16, 24]
    N_SEEDS_B = 3
    BURN_FACTOR_B = 5
    PROD_FACTOR_B = 5

    print(' N fixe : ' + str(N_FIXED))
    print(' kappa_0 : ' + str(KAPPA_LIST))
    print(' Seeds par kappa_0 et par type : ' + str(N_SEEDS_B))
    print(' Total : ' + str(len(KAPPA_LIST) * N_SEEDS_B * 2) + ' simulations')
    print(' Duree estimee : 40-60 minutes')
    print('=' * 75)

    results_B = []
    for kappa in KAPPA_LIST:
        burn = BURN_FACTOR_B * N_FIXED
        prod = PROD_FACTOR_B * N_FIXED
        print('')
        print('--- kappa_0 = ' + str(kappa) + ' (N fixe = ' + str(N_FIXED) + ') -')
        for seed in range(N_SEEDS_B):
            r = run_one(N_FIXED, seed, burn, prod, is_null=False, kappa_0=kappa,
                        results_B.append(r)
        for seed in range(N_SEEDS_B):
            r = run_one(N_FIXED, seed, burn, prod, is_null=True, kappa_0=kappa, l
                        results_B.append(r)

    blob_B = aggregate([r for r in results_B if r['type'] == 'BLOB'], 'kappa_0')
    null_B = aggregate([r for r in results_B if r['type'] == 'NULL'], 'kappa_0')
    ratios_B = compute_ratio_snr(blob_B, null_B)

    print('')
    print('=' * 75)
    print(' RESULTATS PROTOCOLE B')
    print('=' * 75)
    print(' ' + 'kappa_0'.ljust(10) + 'n_tri_blob'.ljust(14) + 'n_tri_null'.ljust
          + 'ratio'.ljust(10) + 'D_blob')
    print(' ' + '-' * 65)

```



```

for kappa in sorted(ratios_B.keys()):
    r = ratios_B[kappa]
    line = ' ' + str(kappa).ljust(10)
    line += '{:.1f}'.format(r['blob_ntri']).ljust(14)
    line += '{:.1f}'.format(r['null_ntri']).ljust(14)
    line += '{:.2f}'.format(r['ratio']).ljust(10)
    line += '{:.4f}'.format(blob_B[kappa]['D_mean'])
    print(line)

print('')
print(' Interpretation :')
print(' Si n_tri_blob CROIT avec kappa_0 : hypothese dilution Rips CONFIRMEE')
print(' Si n_tri_blob DECROIT avec kappa_0 : autre mecanisme')
print(' Si n_tri_blob STABLE : phenomene non-Rips')

return results_B, blob_B, null_B, ratios_B

# =====
# PROTOCOLE C : Test thermalisation a N=50000
# =====

def protocol_C():
    print('')
    print('=' * 75)
    print(' PROTOCOLE C : Test convergence avec burn variable')
    print(' N=50000 fixe, kappa_0=8 fixe, burn in {5N, 10N, 20N}')
    print(' But : verifier que le plateau ~700 n est PAS un artefact thermalisat')
    print('=' * 75)

    N_FIXED = 50000
    BURN_FACTORS = [5, 10, 20]
    N_SEEDS_C = 2
    PROD_FACTOR_C = 5

    print(' N fixe : ' + str(N_FIXED))
    print(' burn factors : ' + str(BURN_FACTORS) + ' x N')
    print(' Seeds par burn factor : ' + str(N_SEEDS_C))
    print(' Total : ' + str(len(BURN_FACTORS) * N_SEEDS_C) + ' simulations Blob')
    print(' Duree estimee : 30-45 minutes')
    print('=' * 75)

    results_C = []
    for bf in BURN_FACTORS:
        burn = bf * N_FIXED
        prod = PROD_FACTOR_C * N_FIXED
        print('')
        print('--- burn_factor = ' + str(bf) + ' (burn = ' + str(burn) + ' pas) -
```

```

        for seed in range(N_SEEDS_C):
            r = run_one(N_FIXED, seed, burn, prod, is_null=False, label_prefix='C')
            r['burn_factor'] = bf
            results_C.append(r)

by_bf = defaultdict(list)
for r in results_C:
    by_bf[r['burn_factor']].append(r)

print('')
print('=' * 75)
print('  RESULTATS PROTOCOLE C')
print('=' * 75)
print('  ' + 'burn_factor'.ljust(14) + 'n_tri_mean'.ljust(14)
      + 'n_tri_std'.ljust(12) + 'drift_mean')
print('  ' + '-' * 60)
for bf in sorted(by_bf.keys()):
    rs = by_bf[bf]
    nt_mean = np.mean([r['ntri_mean'] for r in rs])
    nt_std = np.std([r['ntri_mean'] for r in rs])
    drift_mean = np.mean([r['drift'] for r in rs])
    line = '  ' + (str(bf) + 'N').ljust(14)
    line += '{:.1f}'.format(nt_mean).ljust(14)
    line += '{:.1f}'.format(nt_std).ljust(12)
    line += '{:.3%}'.format(drift_mean)
    print(line)

print('')
print('  Interpretation :')
print('  Si n_tri stable (variation < 5%) : plateau ~700 EST physique [confir')
print('  Si n_tri croit avec burn_factor : sous-thermalisation, plateau a red

return results_C

# =====
# MAIN
# =====
print('=' * 75)
print('  CARACTERISATION DE LA TRANSITION DE REGIME')
print('  Trois protocoles complementaires')
print('=' * 75)
print('  Parametres canoniques : kappa_0=' + str(KAPPA_0_DEFAULT)
      + ', beta=' + str(BETA) + ', tau=' + str(TAU))
print('  Duree totale estimee : 2-3 heures')
print('=' * 75)

```

```

t0 = time.time()

results_A, blob_A, null_A, ratios_A = protocol_A()
results_B, blob_B, null_B, ratios_B = protocol_B()
results_C = protocol_C()

total_time = time.time() - t0

# =====
# RAPPORT FINAL CONSOLIDE
# =====
print('')
print('=' * 75)
print('  RAPPORT FINAL CONSOLIDE')
print('=' * 75)
print('  Temps total : ' + '{:.1f}'.format(total_time / 60) + ' minutes')
print('')

# A : transition
print('  --- PROTOCOLE A (transition) ---')
print('  ' + 'N'.ljust(10) + 'ratio'.ljust(10) + 'n_tri_blob'.ljust(14) + 'D_blob')
for N in sorted(ratios_A.keys()):
    r = ratios_A[N]
    line = '  ' + str(N).ljust(10)
    line += '{:.2f}'.format(r['ratio']).ljust(10)
    line += '{:.1f}'.format(r['blob_ntri']).ljust(14)
    line += '{:.4f}'.format(blob_A[N]['D_mean'])
    print(line)

# Detection automatique de la transition
ratios_list = [(N, ratios_A[N]['ratio']) for N in sorted(ratios_A.keys())]
print('')
print('  Detection automatique de la transition :')
for i in range(1, len(ratios_list)):
    n_prev, r_prev = ratios_list[i-1]
    n_curr, r_curr = ratios_list[i]
    delta = r_curr - r_prev
    rel_delta = abs(delta) / r_prev if r_prev > 0 else 0
    flag = ''
    if rel_delta > 0.10:
        flag = ' <- saut significatif'
    print('    N ' + str(n_prev) + ' -> ' + str(n_curr) + ' : ratio '
          + '{:.2f}'.format(r_prev) + ' -> ' + '{:.2f}'.format(r_curr)
          + ' (delta_rel=' + '{:.1%}'.format(rel_delta) + ')' + flag)

# B : dilution
print('')

```

```

print(' --- PROTOCOLE B (dilution Rips) ---')
print(' ' + 'kappa_0'.ljust(10) + 'ratio'.ljust(10) + 'n_tri_blob')
for kappa in sorted(ratios_B.keys()):
    r = ratios_B[kappa]
    line = ' ' + str(kappa).ljust(10)
    line += '{:.2f}'.format(r['ratio']).ljust(10)
    line += '{:.1f}'.format(r['blob_ntri'])
    print(line)

# Trend analysis B
kappas = sorted(ratios_B.keys())
if len(kappas) >= 2:
    ntri_vals = [ratios_B[k]['blob_ntri'] for k in kappas]
    log_kappa = np.log(kappas)
    log_ntri = np.log(ntri_vals)
    slope, intercept = np.polyfit(log_kappa, log_ntri, 1)
    print('')
    print(' Loi n_tri_blob ~ kappa_0^' + '{:.3f}'.format(slope))
    if slope > 0.5:
        print(' -> Croissance forte : DILUTION RIPS CONFIRMEE')
    elif slope > 0.1:
        print(' -> Croissance moderee : effet partiel')
    elif slope > -0.1:
        print(' -> Pas de dependance : autre mecanisme')
    else:
        print(' -> Decroissance : surprenant, a investiguer')

# C : thermalisation
print('')
print(' --- PROTOCOLE C (thermalisation) ---')
by_bf = defaultdict(list)
for r in results_C:
    by_bf[r['burn_factor']].append(r)
print(' ' + 'burn_factor'.ljust(14) + 'n_tri_mean')
ntri_by_bf = []
for bf in sorted(by_bf.keys()):
    nt_mean = np.mean([r['ntri_mean'] for r in by_bf[bf]])
    ntri_by_bf.append(nt_mean)
    print(' ' + (str(bf) + 'N').ljust(14) + '{:.1f}'.format(nt_mean))

if len(ntri_by_bf) >= 2:
    rel_var = (max(ntri_by_bf) - min(ntri_by_bf)) / min(ntri_by_bf)
    if min(ntri_by_bf) > 0:
        print(' Variation relative entre burn_factor min/max : ' + '{:.1%}'.format(rel_var))
    if rel_var < 0.05:
        print(' -> Plateau STABLE : thermalisation confirmee, plateau physique [')
    elif rel_var < 0.15:
        print(' -> Plateau quasi-stable : thermalisation probable')

```

```

        else:
            print(' -> Plateau instable : sous-thermalisation, augmenter burn_factor

print('')
print('=' * 75)
print(' CONCLUSION SCIENTIFIQUE AUTOMATIQUE')
print('=' * 75)

# Conclusion synthetique
verdict_a = 'transition observee' if any(abs(ratios_list[i][1] - ratios_list[i-1]
print(' Protocole A : ' + verdict_a)

if len(kappas) >= 2:
    if slope > 0.5:
        verdict_b = 'dilution Rips confirmee'
    elif slope > 0.1:
        verdict_b = 'effet partiel kappa_0'
    else:
        verdict_b = 'mecanisme non-Rips'
    print(' Protocole B : ' + verdict_b)

if len(ntri_by_bf) >= 2:
    if rel_var < 0.05:
        verdict_c = 'plateau physique confirme [M FORT]'
    elif rel_var < 0.15:
        verdict_c = 'plateau probable'
    else:
        verdict_c = 'sous-thermalisation, plateau a redefinir'
    print(' Protocole C : ' + verdict_c)

print('')
print(' Si A=transition + B=dilution Rips + C=plateau confirme :')
print(' -> Decouverte completement caracterisee, mecanisme identifie [M MAJEUR]'
print(' -> B3 [FAIL] remplace par mecanisme physique distinctif du Blob')
print(' -> Candidat fort pour mecanisme de fin d inflation')
print('')
print('=' * 75)
print(' FIN DU RAPPORT')
print('=' * 75)

```